Ticketflow — Day 2

One seat, fifty people, and the gap between reading and writing

> **Today is one endpoint.** `POST /api/bookings` — take a seat, give back a booking. You will write it in about fifteen minutes and it will look right.
>
> The rest of the day is finding out that it is not, watching exactly how it fails, and fixing it with one line of SQL. Then a test that proves the fix, which is the single most useful thing on this whole project.

Today's plan

PART	WHAT HAPPENS
1	Write booking the obvious way. It works when you test it by hand.
2	Fifty people, one seat. Two different disasters, both measured.
3	`SELECT ... FOR UPDATE`. One line. 1 winner, 49 clean refusals, every run.
4	Cancel — a transaction running backwards.
5	Bookings list — a JOIN across all three tables.
6	The test that proves it, and the one you will talk about in interviews.

About seven hours, and roughly two of those are Part 2 and 3, which is twenty minutes of typing and a lot of sitting with it. That is the right ratio. Do not rush that bit — it is the only part of this project an interviewer will dig into.

PART 1 · Booking a seat, the obvious way

What has to happen when somebody books seat 14:

in English

```
1. find the seat
2. if it does not exist      -> 404
3. if it is already booked   -> 409
4. mark it booked
5. store the booking row
```

Steps 4 and 5 are two writes that must both happen — a seat marked booked with no booking row is a chair nobody can ever sit in. You met that yesterday, so it goes in a transaction.

409 is the one new status code. **Conflict**: your request was well formed and you are allowed to make it, but the current state of the world says no. That is different from 400 (you sent nonsense) and from 404 (there is no such thing).

The repository

app/repositories/bookings.py

```
async def get_seat(conn, seat_id):
    return await conn.fetchrow("SELECT * FROM seats WHERE id = $1", seat_id)


async def mark_booked(conn, seat_id):
    await conn.execute("UPDATE seats SET status = 'booked' WHERE id = $1", seat_id)


async def insert(conn, seat_id, user_email):
    return await conn.fetchrow(
        """INSERT INTO bookings (seat_id, user_email)
           VALUES ($1, $2)
           RETURNING *""",
        seat_id, user_email,
    )
```

Three functions, nothing you have not written before.

The service

app/services/bookings.py

```
from app.repositories import bookings as repo


class SeatNotFound(Exception):
    pass


class SeatTaken(Exception):
    pass


async def create(pool, seat_id, user_email):
    seat = await repo.get_seat(pool, seat_id)
    if seat is None:
        raise SeatNotFound()
    if seat["status"] == "booked":
        raise SeatTaken()

    async with pool.acquire() as conn:
        async with conn.transaction():
            await repo.mark_booked(conn, seat_id)
            booking = await repo.insert(conn, seat_id, user_email)
    return booking
```

Read it against the five English steps. Find the seat, two guards, then the two writes inside yesterday's fence. It matches line for line.

The router

app/routers/bookings.py

```
from fastapi import APIRouter, HTTPException, Request

from app.schemas import BookingIn
from app.services import bookings as service

router = APIRouter(prefix="/api/bookings")


@router.post("", status_code=201)
async def create_booking(body: BookingIn, request: Request):
    try:
        row = await service.create(request.app.state.pool, body.seat_id, body.user_email)
    except service.SeatNotFound:
        raise HTTPException(status_code=404, detail="Seat not found")
    except service.SeatTaken:
        raise HTTPException(status_code=409, detail="Seat already booked")
    return dict(row)
```

Two `except` blocks instead of one. Same translation move, one exception per status code.

main.py — the two lines that change

```
from app.routers import bookings, events
...
app.include_router(events.router)
app.include_router(bookings.router)
```

Try it

measured

```
POST   /api/bookings  seat 1       201   {"id":587,"seat_id":41,"user_email":"raj@kgp.ac.in",...}
POST   /api/bookings  seat 1 again 409   {"detail":"Seat already booked"}
POST   /api/bookings  seat 99999   404   {"detail":"Seat not found"}
GET    /api/events/1/seats?status=available   -> seat_no 2, 3
```

Correct, every time. Book a seat, it goes. Book it again, refused. The seat list updates.

It is also broken, and you cannot see it from here, because you are one person clicking one button.

PART 2 · Fifty people, one seat

Make an event with exactly one seat. Then send fifty booking requests for that seat at the same instant — not one after another, all at once, the way a hall of students hits the page when booking opens.

I ran it three times. Here is what came back:

measured — your code, exactly as written in Part 1

```
201s  1 | 409s  6 | crashes 43 | booking rows for that seat: 1
201s  1 | 409s 45 | crashes  4 | booking rows for that seat: 1
201s  1 | 409s 13 | crashes 36 | booking rows for that seat: 1
```

Two things to notice, and the second one is the real story.

The data is fine. One booking row, every run. Nobody got double-sold. Good.

Forty-three people got a 500. Not a “sorry, taken” page — a crash. Your server threw an unhandled exception in their face. And the number is different every run: 43, then 4, then 36. You cannot reproduce it, you cannot test for it by hand, and it only happens when it matters, which is the moment lots of people want the same thing.

WHY THE CRASH

Walk two requests through your code, seat free, arriving together.

A reads the seat: `'available'`. Before A writes, B reads the seat: also `'available'`, because A has not written yet. Both pass the `SeatTaken` guard. Both go into the transaction. Both try to insert a booking row.

The second one hits `UNIQUE (seat_id)` and Postgres refuses it. Your service was not expecting that — it already checked, remember — so `UniqueViolationError` sails past every `except` block you wrote and out to the user as a 500.

Your check was true when you made it, and false by the time you acted on it. That is a **lost update**, and you have met it twice already: the visit counter in project 3, and “is this short code taken?” before that.

So what was the constraint worth?

Everything, as it turns out. Here is the identical code with `UNIQUE (seat_id)` dropped from the table — same endpoint, same fifty people:

measured — same code, constraint removed

```
201s 41 | 409s  9 | crashes 0 | booking rows for that seat: 41
201s 41 | 409s  9 | crashes 0 | booking rows for that seat: 41
201s 40 | 409s 10 | crashes 0 | booking rows for that seat: 40
```

Forty-one people holding a ticket for the same chair. Forty-one 201s. Nothing crashed, nothing logged, no error anywhere. Your API told forty-one people "yes, seat 14 is yours" and meant it.

That is what yesterday's one line in `schema.sql` was standing between you and. It did not fix your code — your code is exactly as wrong as it was. It stopped the wrongness reaching the data.

Which leaves you with a clear picture of where you actually are:

	DATA CORRECT?	USERS HAPPY?
your code, no `UNIQUE`	no — 41 bookings, one seat	yes, horribly
your code, with `UNIQUE`	yes	no — dozens of 500s
what you want	yes	yes — one 201, the rest a clean 409

The constraint got you the first column. The second column is the rest of today.

PART 3 · Closing the gap

The problem is not the check. The check is correct. The problem is the **gap** — the time between you reading the seat and you writing it, during which somebody else can read the same seat and reach the same conclusion.

So you need a way to say: *from the moment I read this row until I am finished with it, nobody else reads it.*

Postgres has exactly that, and it is two words on the end of your `SELECT`.

app/repositories/bookings.py

```
async def get_seat_locked(conn, seat_id):
    return await conn.fetchrow(
        "SELECT * FROM seats WHERE id = $1 FOR UPDATE", seat_id
    )
```

WHAT FOR UPDATE DOES

It reads the row **and locks it**, in one step.

Any other transaction that tries to `SELECT ... FOR UPDATE` that same row does not fail and does not get stale data — it **waits**. It sits there until your transaction ends. Then it wakes up and reads the row *as you left it*.

So request B does not read `'available'` any more. It waits for A to commit, and then reads `'booked'` — which is the truth. Its guard fires, and it returns a clean 409.

The lock is released when the transaction ends. There is nothing to unlock by hand. This is why the `SELECT` has to be *inside* the transaction — outside one, there is no "until" for the lock to last for.

The one change to your service

Yesterday the transaction wrapped only the two writes. Now it has to wrap the read as well — because the read is the thing being protected:

app/services/bookings.py

```
async def create(pool, seat_id, user_email):
    async with pool.acquire() as conn:
        async with conn.transaction():
            seat = await repo.get_seat_locked(conn, seat_id)
            if seat is None:
                raise SeatNotFound()
            if seat["status"] == "booked":
                raise SeatTaken()

            await repo.mark_booked(conn, seat_id)
            booking = await repo.insert(conn, seat_id, user_email)
    return booking
```

Compare it to Part 1 line by line. The guards are identical. The writes are identical. **Four lines moved inside the fence, and the `SELECT` gained two words.** That is the whole fix.

Raising `SeatNotFound` from inside the transaction is fine, and correct — the exception rolls the transaction back on its way out, which is exactly what should happen when you are abandoning the booking.

Same fifty people, same one seat:

measured

```
201s  1 | 409s 49 | crashes 0 | booking rows for that seat: 1
201s  1 | 409s 49 | crashes 0 | booking rows for that seat: 1
201s  1 | 409s 49 | crashes 0 | booking rows for that seat: 1
```

One winner. Forty-nine people told politely that the seat is gone. Nobody sees an error page. Identical every single run — and that is the part that matters: the old numbers moved around because the outcome depended on timing. These do not move, because it no longer does.

PART 4 · Cancel — the transaction running backwards

Cancelling is booking in reverse, and it is two writes again: delete the booking row, and put the seat back to `'available'`. Half of that is a disaster in either direction — a deleted booking with a still-booked seat is a chair nobody can buy, and a freed seat with a live booking row is a seat sold twice.

app/repositories/bookings.py

```
async def mark_available(conn, seat_id):
    await conn.execute("UPDATE seats SET status = 'available' WHERE id = $1", seat_id)


async def get(conn, booking_id):
    return await conn.fetchrow("SELECT * FROM bookings WHERE id = $1", booking_id)


async def delete(conn, booking_id):
    return await conn.execute("DELETE FROM bookings WHERE id = $1", booking_id)
```

app/services/bookings.py

```
async def cancel(pool, booking_id):
    async with pool.acquire() as conn:
        async with conn.transaction():
            booking = await repo.get(conn, booking_id)
            if booking is None:
                raise BookingNotFound()
            await repo.delete(conn, booking_id)
            await repo.mark_available(conn, booking["seat_id"])
```

You need the booking row before deleting it, because it is the only thing that knows which seat to free. Read it, delete it, free the seat — all three inside one fence.

No `FOR UPDATE` here, and it is worth knowing why rather than just noticing: two people cancelling the same booking is not a race you can lose. The second `DELETE` removes nothing and the seat ends up available either way. Racing only hurts when the two requests would reach *different* conclusions, and here they do not.

app/routers/bookings.py

```
@router.delete("/{booking_id}", status_code=204)
async def cancel_booking(booking_id: int, request: Request):
    try:
        await service.cancel(request.app.state.pool, booking_id)
    except service.BookingNotFound:
        raise HTTPException(status_code=404, detail="Booking not found")
```

measured

```
event with 2 seats, book seat 1
  available seats -> [2]
DELETE /api/bookings/587            204
  available seats -> [1, 2]
DELETE /api/bookings/587 again      404
```

PART 5 · The bookings list — three tables at once

A booking row holds a `seat_id` and an email. Useful to the database, useless to a human: nobody wants to be told they have booked seat id 41. They want *seat 14 for Spring Fest*.

Seat number is in `seats`. Event name is in `events`. Yesterday you joined two tables; this is the same move, twice:

app/repositories/bookings.py

```
async def list_all(conn):
    return await conn.fetch(
        """SELECT b.id, b.user_email, b.created,
                  s.seat_no,
                  e.name AS event_name
           FROM bookings b
           JOIN seats  s ON s.id = b.seat_id
           JOIN events e ON e.id = s.event_id
           ORDER BY b.id"""
    )
```

Read the joins as a chain: a booking knows its seat, that seat knows its event. Each `ON` is one link, and by the last line Postgres has all three rows side by side and you pick the columns you want.

Plain `JOIN` this time, not `LEFT`. Yesterday `LEFT` was there to keep events with no matching seats. Here a booking without a seat cannot exist — the foreign key guarantees it — so there is nothing to keep.

And the `SELECT` lists columns instead of `*`, because three tables joined together have three `id` columns and you only want one of them.

measured

```
GET /api/bookings
[{"id":587,"user_email":"raj@kgp.ac.in","created":"...",
  "seat_no":1,"event_name":"Illumination"}]
```

PART 6 · The test that proves it

Everything you have measured today, I measured. You should be able to measure it too, on demand, in one second — and that is a test.

This matters more than it sounds. A concurrency bug cannot be found by clicking. If you ever touch this service again, the only thing standing between you and silently reintroducing the bug is a test that fires fifty requests and counts.

```
pip install pytest pytest-asyncio
```

pytest.ini, next to main.py

```
[pytest]
asyncio_mode = auto
```

tests/test_booking.py

```
import asyncio

import httpx
import pytest

from main import app


async def make_event(client, seats):
    r = await client.post("/api/events", json={
        "name": "Test Event", "venue": "Test Venue",
        "starts_at": "2026-12-01T10:00:00Z", "total_seats": seats,
    })
    event_id = r.json()["id"]
    seat_rows = (await client.get(f"/api/events/{event_id}/seats")).json()
    return event_id, seat_rows
```

A small helper, because both tests need an event with seats. Nothing clever — it calls your own API the way any client would.

```
@pytest.mark.asyncio
async def test_one_seat_one_winner():
    transport = httpx.ASGITransport(app=app)
    async with httpx.AsyncClient(transport=transport, base_url="http://test") as client:
        async with app.router.lifespan_context(app):
            event_id, seats = await make_event(client, 1)
            seat_id = seats[0]["id"]

            responses = await asyncio.gather(*(
                client.post("/api/bookings",
                            json={"seat_id": seat_id, "user_email": f"p{i}@test.com"})
                for i in range(50)
            ))

            codes = [r.status_code for r in responses]
            assert codes.count(201) == 1
            assert codes.count(409) == 49

            stored = (await client.get("/api/bookings")).json()
            assert len([b for b in stored if b["seat_no"] == 1]) == 1

            await client.delete(f"/api/events/{event_id}")
```

LINE	WHAT IT IS FOR
`ASGITransport(app=app)`	Calls your app directly, in-process. No server to start, no port, no network.
`lifespan_context(app)`	Runs your startup code so `app.state.pool` exists. Without it there is no pool and everything fails.
`asyncio.gather(*(...))`	The whole point. Fires all fifty at once instead of one after another — this is what makes it a race.
the three `asserts`	One winner, forty-nine refusals, one row in the database. Remove `FOR UPDATE` and all three fail.

tests/test_booking.py — second test, same two setup lines as the first

```
@pytest.mark.asyncio
async def test_cancel_frees_the_seat():
    ...   # same two "async with" lines as above
            booking = (await client.post("/api/bookings",
                       json={"seat_id": seat_id, "user_email": "a@test.com"})).json()

            left = (await client.get(f"/api/events/{event_id}/seats?status=available")).json()
            assert len(left) == 1

            await client.delete(f"/api/bookings/{booking['id']}")

            left = (await client.get(f"/api/events/{event_id}/seats?status=available")).json()
            assert len(left) == 2
```

measured

```
pytest -q

..                                                       [100%]
2 passed in 1.10s
```

> **Go and break it on purpose, once.** Take `FOR UPDATE` out of that one query and run `pytest` again. Watch the first test fail with something like `assert 7 == 1`.
>
> That is the moment the test earns its keep, and it is also the moment you will be able to describe in an interview without rehearsing it.

PART 7 · Break it on purpose

DO THIS	AND NOTICE
Remove `FOR UPDATE` and run `pytest`	The count assertion fails, and with a different number each run. Put it back.
Remove `FOR UPDATE` and book by hand, ten times	Perfect every time. This is why the test exists and clicking does not.
Drop `UNIQUE (seat_id)` and remove `FOR UPDATE` together	Dozens of 201s for one seat. Both guards gone, nothing left.
Keep `FOR UPDATE` but move the `SELECT` outside the transaction	Back to crashes. The lock needs a transaction to last for.
In `psql`: `BEGIN; SELECT * FROM seats WHERE id=1 FOR UPDATE;` and leave it open, then book that seat	Your request hangs. That is the waiting, visible. Type `COMMIT;` and watch it finish.
Cancel a booking, then book the same seat again	201. The cancel really did free it.
Delete an event that has bookings	Gone — seats and bookings with it. Yesterday's cascade, two levels deep.

Where this leaves you

YOU CAN NOW	WHICH IS
Group writes so they cannot be half-done	a transaction
Stop two requests deciding the same thing	a row lock
Tell a lost update from an ordinary bug	the thing most backend bugs actually are
Read across three tables in one query	a join chain
Write a test that proves a race is fixed	the part of this project worth talking about
Pick 400 / 404 / 409 correctly	saying the right thing to the user

What is left

Three things, and none of them need a document — we do them in chat over a couple of evenings, because they fail in ways that depend on your machine and your account, and a PDF would be guessing:

A validation and error pass — a couple of hours going back through every endpoint asking what a hostile user could send it.

Deploy — three hours, and most of it will be environment variables fighting you. A live URL is what turns this from homework into a thing that exists.

The README — not a feature list. The race condition, what the naive version did, what `FOR UPDATE` changed, with the measured numbers. That document is your interview answer, written down before anyone asks for it. It is also where I will give you the two sentences on "why `FOR UPDATE` and not something else," now that it works.

Three lines for your notes

```
a check is only true until you stop holding the row
FOR UPDATE makes the other request wait, not fail
constraints protect the data; the lock protects the user
```